RAJALAKSHMI ENGINEERING COLLEGE

P Saam 251801276-2025-AIDS

P2

Started on	Thursday, 27 August 2026, 8:55 AM
State	Finished
Completed on	Thursday, 27 August 2026, 8:56 AM
Time taken	1 min 3 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor g[i], which is the minimum size of a cookie that the child will be content with; and each cookie j has a size s[j]. If s[j] >= g[i], we can assign the cookie j to the child i, and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

Example 1:

Input:

3

1 2 3

2

1 1

Output:

1

Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3.

And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content.

You need to output 1.

Constraints:

1 <= g.length <= 3 * 10^4

0 <= s.length <= 3 * 10^4

1 <= g[i], s[j] <= 2^31 - 1

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

// Comparison function for sorting integers with qsort
int compare(const void* a, const void* b) {
    int int_a = *((int*)a);
    int int_b = *((int*)b);
    if (int_a < int_b) return -1;
    if (int_a > int_b) return 1;
    return 0;
}

int findContentChildren(int* g, int gSize, int* s, int sSize) {
    // Sort both the children's greed factors and cookie sizes in ascending order
    qsort(g, gSize, sizeof(int), compare);
    qsort(s, sSize, sizeof(int), compare);

    int child = 0;
    int cookie = 0;

    // Traverse through both arrays using two pointers
    while (child < gSize && cookie < sSize) {
        // If the current cookie can satisfy the current child
        if (s[cookie] >= g[child]) {
            child++; // Move to the next child
        }
        // Regardless of whether it matched or was too small,
        // move to the next cookie
        cookie++;
    }

    // The index of the child pointer represents the number of content children
    return child;
}

int main() {
    int gSize, sSize;

    // Reading input format based on standard competitive programming layout
    if (scanf("%d", &gSize) != 1) return 0;
    int* g = (int*)malloc(gSize * sizeof(int));
    for (int i = 0; i < gSize; i++) {
        scanf("%d", &g[i]);
    }
```

```
45
46  if (scanf("%d", &sSize) != 1) {
47      free(g);
48      return 0;
49  }
50  int* s = (int*)malloc(sSize * sizeof(int));
51  for (int i = 0; i < sSize; i++) {
52      scanf("%d", &s[i]);
```

	Input	Expected	Got	
✔	2 1 2 3 1 2 3	2	2	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course